Bölüm 13 Soyut Sınıflar ve Arayüzler



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Motivasyon

- GUI arayüzü için arayüzler hakkında bilgi sahibi olmanız gerekir. Arayüz bir arabirim sınıflar için ortak davranış tanımlamak içindir (ilgisiz sınıflar dahil). Arayüzlerden önce, ilk olarak: soyut sınıflar.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

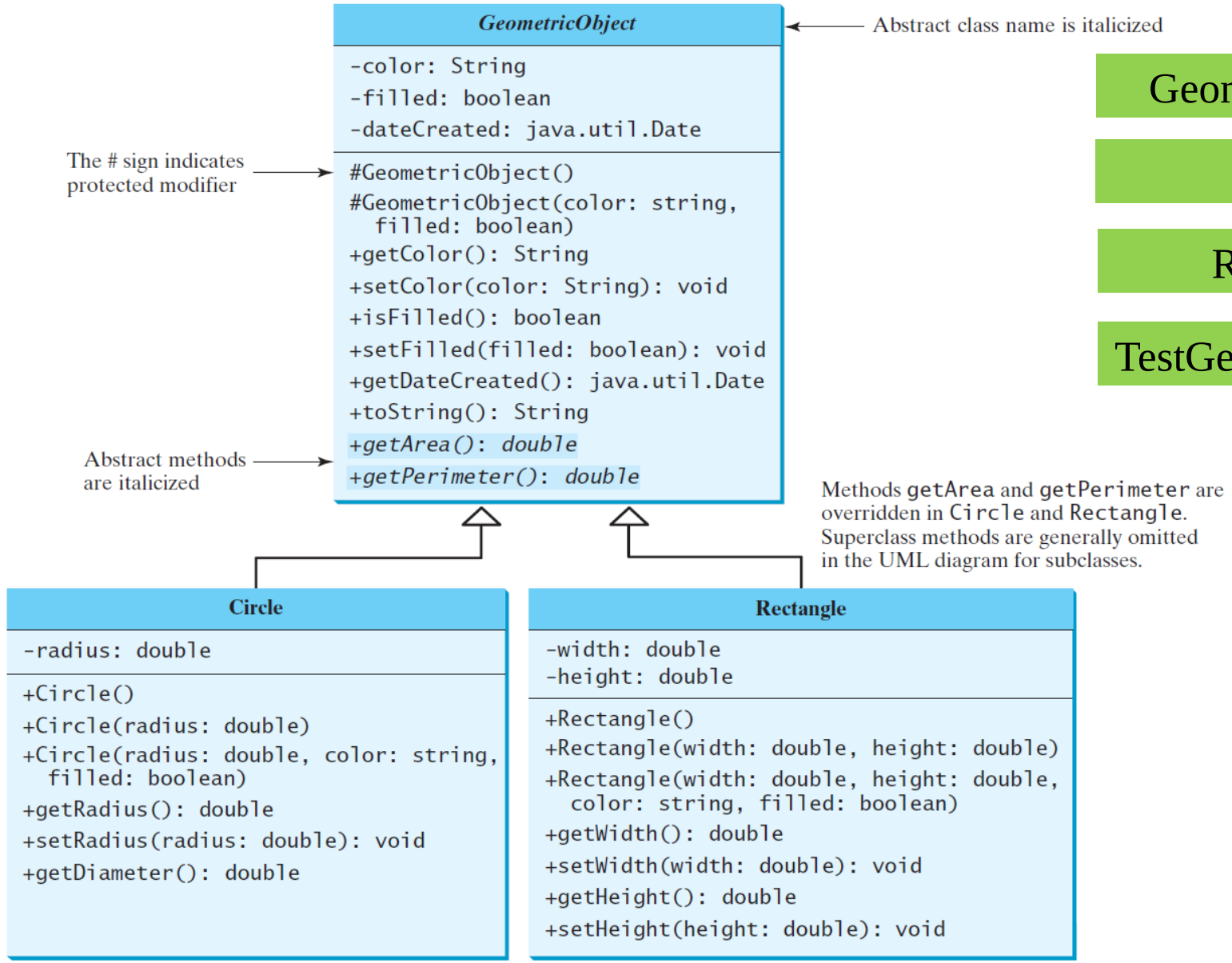
Amaçlar

- Soyut sınıflar tasarlamak ve kullanmak(§13.2).
- Soyut Number sınıfını (§13.3) kullanarak numerik wrapper sınıflarını, BigInteger ve BigDecimal'i genelleştirmek.
- Calendar ve GregorianCalendar sınıflarını kullanarak bir takvimi oluşturmak(§13.4).
- Arayüz kullanan nesneler için ortak davranış belirlemek(§13.5).
- Arayüzleri tanımlamak ve arayüzleri uygulayan sınıfları tanımlamak (§13.5).
- Compareble arayüzü kullanarak doğal bir düzen tanımlamak (§13.6).
- Cloneable arayüzü kullanarak nesneleri klonlanabilir yapmak (§13.7).
- Somut sınıflar, soyut sınıflar ve arayüzler arasındaki benzerlik ve farklılıkları araştırmak (§13.8).
- Rasyonel sayıları işlemek için Rational sınıfını tasarlamak (§13.9).
- Sınıf tasarım yönergelerine uygun sınıflar tasarlamak (§13.10).



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Soyut Sınıflar ve Soyut Metotlar



GeometricObject

Circle

Rectangle

TestGeometricObject

Run

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Soyut Sınıfta Soyut Yöntem

Soyut bir yöntem soyut olmayan bir sınıfta yer alamaz. Bir soyut üst sınıfın, bir alt sınıfı tüm soyut yöntemleri uygulamazsa, alt sınıfın soyut olarak tanımlanması gerekir. Başka bir deyişle, soyut bir sınıftan genişletilen soyut olmayan bir alt sınıfta, alt sınıfta kullanılmasalar bile, tüm soyut yöntemler uygulanmalıdır.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Soyut Sınıftan Nesne Oluşturulamaz

Soyut bir sınıf, new operatörü kullanılarak instantiate(başlatılamaz) edilemez, ancak constructor tanımlayabilirsiniz. Alt sınıflarının constructurları tarafından çağrılabilir. Örneğin, GeometricObject constrcutorları Circle sınıfında ve Rectangle sınıfında çağrılır.



Soyut Yöntem Olmadan Soyut Sınıf

Soyut yöntemler içeren bir sınıf soyut olmalıdır. Ancak, soyut yöntem içermeyen soyut bir sınıf tanımlamak mümkündür. Bu durumda, new operatörü kullanarak sınıfın örneklerini oluşturamazsınız (instance). Bu sınıf, yeni bir alt sınıf tanımlamak için temel sınıf olarak kullanılır.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Soyut Sınıfın Super Sınıfı Somut(concrete) olabilir

Bir alt sınıf (subclass), üst sınıfı (superclass) somut olsa bile soyut olabilir. Örneğin, Object sınıfı somuttur, ancak GeometricObject gibi alt sınıfları soyut olabilir.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Soyut Olarak Geçersiz Kılınan (override) Somut Yöntem

Bir alt sınıf, soyut tanımlamak için bir yöntemi üst sınıfından geçersiz kılabilir. Bu nadirdir, ancak üst sınıftaki yöntemin uygulanması alt sınıfta geçersiz olduğunda yararlıdır. Bu durumda, alt sınıf soyut olarak tanımlanmalıdır.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

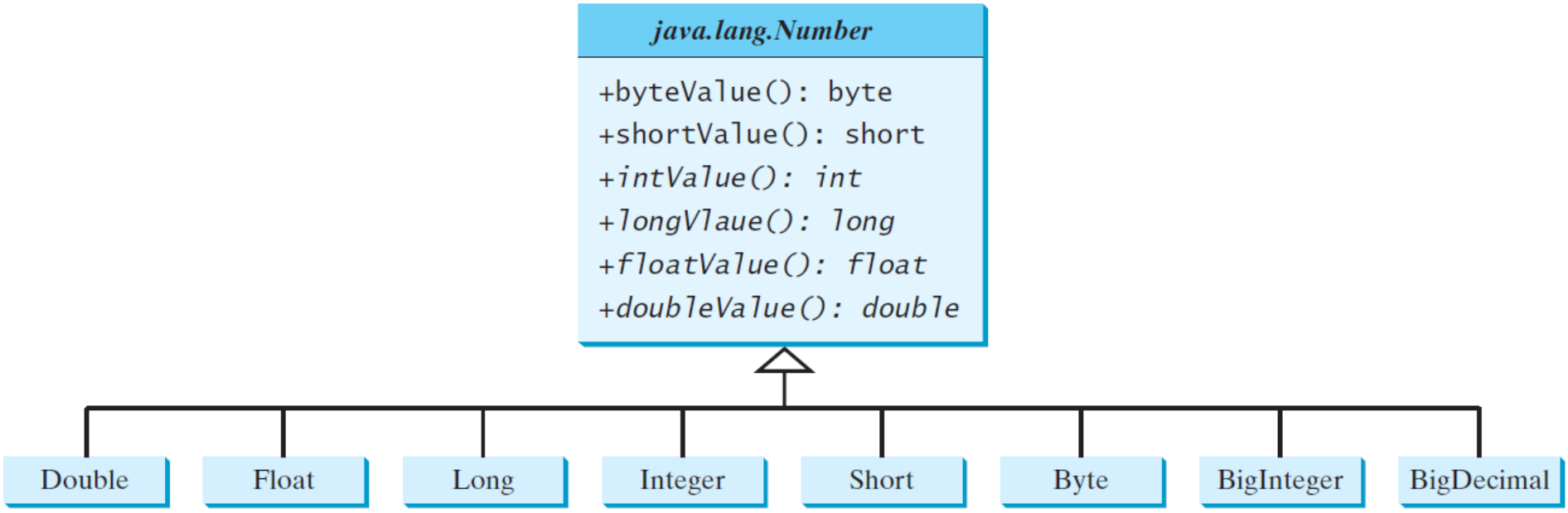
tür olarak soyut sınıf

New işleci kullanarak soyut bir sınıftan örnek oluşturamazsınız, ancak soyut bir sınıf veri türü olarak kullanılabilir. Bu nedenle, öğeleri GeometricObject türünde olan bir dizi oluşturan aşağıdaki ifade doğrudur.

GeometricObject[] geo = new GeometricObject[10];



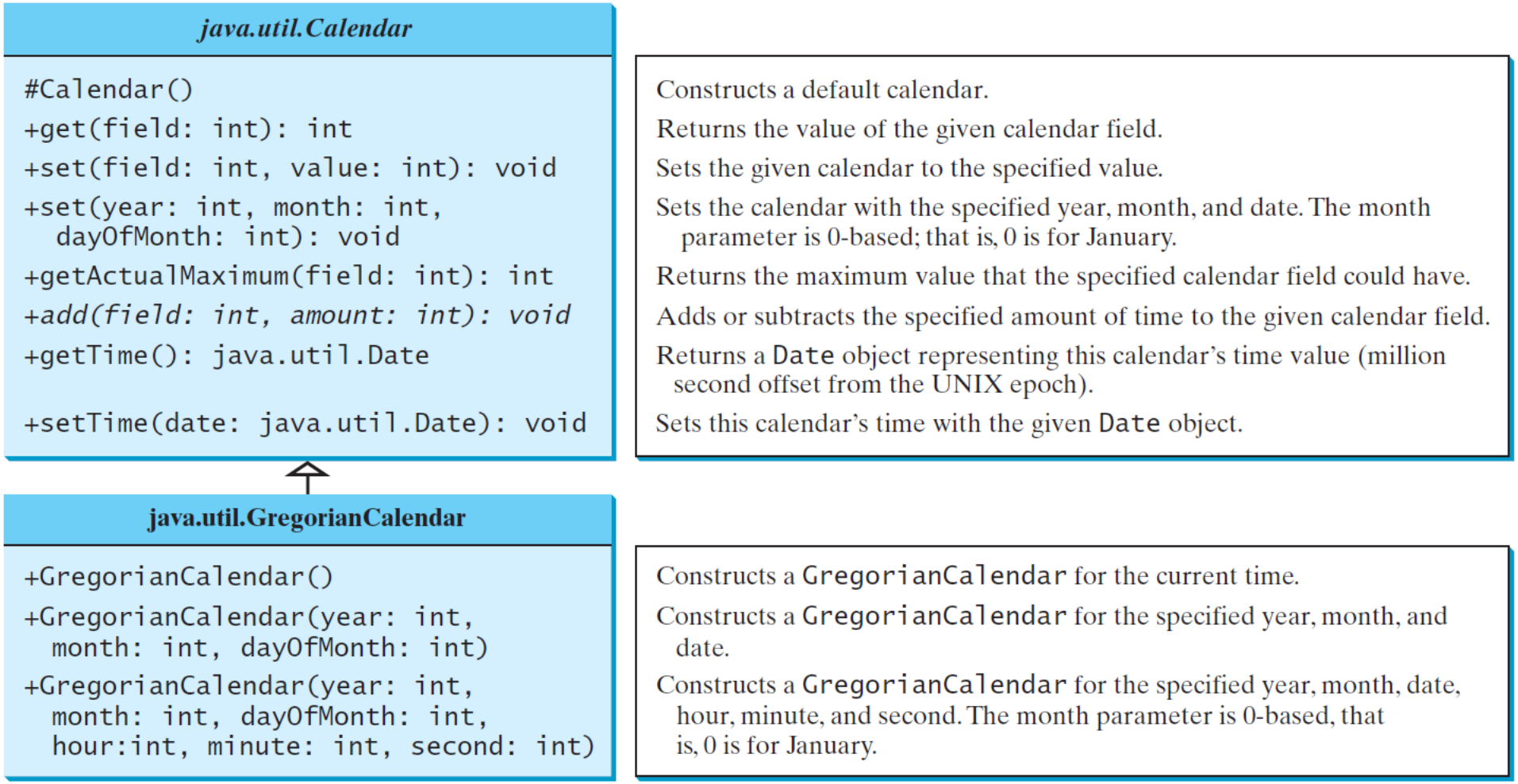
Örnek Olay: Soyut Sayı Sınıfı



LargestNumbers

Run

Soyut Calendar Sınıfı ve GregorianCalendar alt sınıfı



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Soyut Calendar Sınıfı ve GregorianCalendar alt sınıfı

Java.util.Date örneği, milisaniye hassasiyetinde belirli bir anı temsil eder. java.util.Calendar, Date nesnesinden yıl, ay, tarih, saat, dakika ve saniye gibi ayrıntılı bilgileri ayıklamak için kullanılan temel bir sınıftır. Takvim'in alt sınıfları Gregorian calendar, Lunar Calendar ve Jewish Calendar gibi belirli takvim sistemlerini implemente edebilir. Şu anda, GregorianCalendar için java.util.GregorianCalendar Java API'sında desteklenmektedir.



GregorianCalendar Sınıfı

Geçerli saatle, varsayılan bir GregorianCalendar oluşturmak için yeni GregorianCalendar () ve belirtilen yıl, ay ve tarih ile bir GregorianCalendar oluşturmak için yeni GregorianCalendar (yıl, ay, tarih) kullanabilirsiniz. Month parametresi 0 tabanlıdır, yani 0 Ocak içindir.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Calendar Sınıfında get metodu

Calendar sınıfında tanımlanan get (int field) yöntemi, bir Calendar nesnesinden tarih ve saat bilgilerini ayıklamak için kullanışlıdır. Alanlar, aşağıda gösterildiği gibi sabitler olarak tanımlanır.

Constant	*Description*
YEAR	The year of the calendar.
MONTH	The month of the calendar, with 0 for January.
DATE	The day of the calendar.
HOUR	The hour of the calendar (12-hour notation).
HOUR_OF_DAY	The hour of the calendar (24-hour notation).
MINUTE	The minute of the calendar.
SECOND	The second of the calendar.
DAY_OF_WEEK	The day number within the week, with 1 for Sunday.
DAY_OF_MONTH	Same as DATE.
DAY_OF_YEAR	The day number in the year, with 1 for the first day of the year.
WEEK_OF_MONTH	The week number within the month, with 1 for the first week.
WEEK_OF_YEAR	The week number within the year, with 1 for the first week.
AM_PM	Indicator for AM or PM (0 for AM and 1 for PM).



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Calendar Sınıfından Tarih / Saat Bilgilerini Alma

TestCalendar

Run



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Arayüzler

Arayüz nedir?

Bir arayüz neden yararlıdır?

Bir arayüzü nasıl tanımlarsınız?

Bir arayüzü nasıl kullanıyorsunuz?



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Arayüz nedir? Bir arayüz neden yararlıdır?

Bir arayüz, yalnızca sabitleri ve soyut yöntemleri içeren, sınıf benzeri bir yapıdır. Birçok yönden, arayüz soyut bir sınıfa benzer, ancak arayüzün amacı nesneler için ortak davranış belirtmektir. Örneğin, nesnelerin uygun arayüzler kullanılarak karşılaştırılabilir, yenilebilir, klonlanabilir olduğunu belirtebilirsiniz.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Arayüz Tanımlama

Bir arayüzü bir sınıftan ayırmak için Java bir arayüzü tanımlamak için aşağıdaki sözdizimini kullanır:

```
public interface InterfaceName {
  constant declarations;
  abstract method signatures;
}
```

Örnek:

```
public interface Edible {
  /** Describe how to eat */
  public abstract String howToEat();
}
```



Arayüz Özel Bir Sınıftır

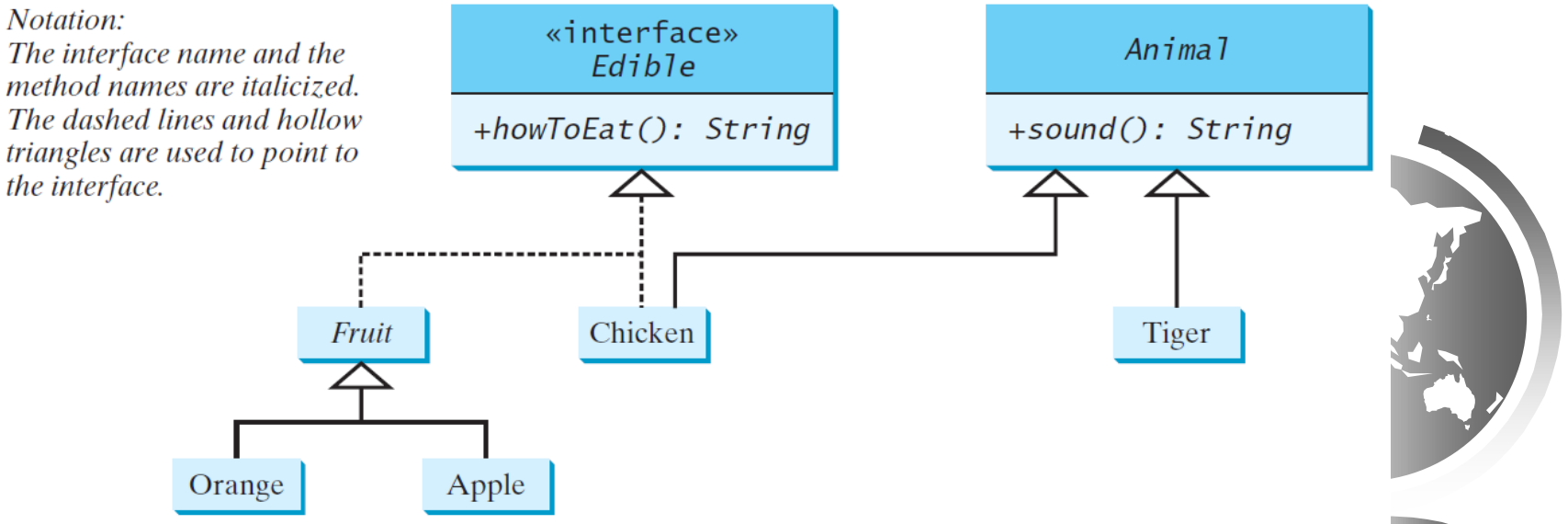
Bir arayüz Java'da özel bir sınıf gibi ele alınır. Her arayüz, tıpkı normal bir sınıf gibi ayrı bir bayt kodu dosyasında derlenir. Soyut bir sınıf gibi, new işleci kullanarak bir arayüzden bir örnek oluşturamazsınız, ancak çoğu durumda soyut bir sınıfı kullandığınız şekilde bir arayüzü az çok kullanabilirsiniz. Örneğin, casting işleminin sonucu olarak bir değişkeni veri türü olarak bir arayüz kullanabilir.



Örnek

Artık bir nesnenin Edible olup olmadığını belirtmek için Edible arayüzü kullanabilir. Bu, nesnenin sınıfının implements anahtar sözcüğünü kullanarak bu arayüzü uygulamasına izin verilerek gerçekleştirilir. Örneğin, Chicken ve Apple sınıfları Edible arabirimi uygular (Bkz. TestEdible).

Edible | TestEdible | Run



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Arabirimlerde Değiştiricilerin (modifier) İhmal Edilmesi

Tüm veri alanları public final static'tir ve tüm yöntemler bir arayüzde public abstract'tır. Bu nedenle, aşağıda gösterildiği gibi bu değiştiriciler ihmal edilebilir:

```
public interface T1 {
  public static final int K = 1;

  public abstract void p();
}
```

Equivalent

```
public interface T1 {
  int K = 1;

  void p();
}
```

Bir arayüzde tanımlanan bir sabite, InterfaceName.CONSTANT_NAME sözdizimi (örn., T1.K) kullanılarak erişilebilir.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Örnek: The Comparable Arayüzü

```
// This interface is defined in
// java.lang package
package java.lang;

public interface Comparable<E> {
  public int compareTo(E o);
}
```



toString, equals, and hashCode Metotları

Her wrapper (sarmalayıcı) sınıfı, Object sınıfında tanımlanan toString, equals ve hashCode yöntemlerini override (geçersiz) eder. Tüm nümerik wrapper sınıfları ve Character sınıfı Comparable arayüzünü implemente edebilir, compareTo metotu bu sınıflarda implemente edilmiştir.



Integer and BigInteger Sınıfları

```
public class Integer extends Number
    implements Comparable<Integer> {
  // class body omitted

  @Override
  public int compareTo(Integer o) {
    // Implementation omitted
  }
}
```

```
public class BigInteger extends Number
    implements Comparable<BigInteger> {
  // class body omitted

  @Override
  public int compareTo(BigInteger o) {
    // Implementation omitted
  }
}
```

String and Date Sınıfları

```
public class String extends Object
    implements Comparable<String> {
  // class body omitted

  @Override
  public int compareTo(String o) {
    // Implementation omitted
  }
}
```

```
public class Date extends Object
    implements Comparable<Date> {
  // class body omitted

  @Override
  public int compareTo(Date o) {
    // Implementation omitted
  }
}
```

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Örnek

```
1  System.out.println(new Integer(3).compareTo(new Integer(5)));
2  System.out.println("ABC".compareTo("ABE"));
3  java.util.Date date1 = new java.util.Date(2013, 1, 1);
4  java.util.Date date2 = new java.util.Date(2012, 1, 1);
5  System.out.println(date1.compareTo(date2));
```



Generic `sort` metotu

n bir Integer nesnesi, s bir String nesnesi ve d bir Date nesnesi olsun. Aşağıdaki ifadelerin tümü doğrudur.

```
n instanceof Integer
n instanceof Object
n instanceof Comparable
```

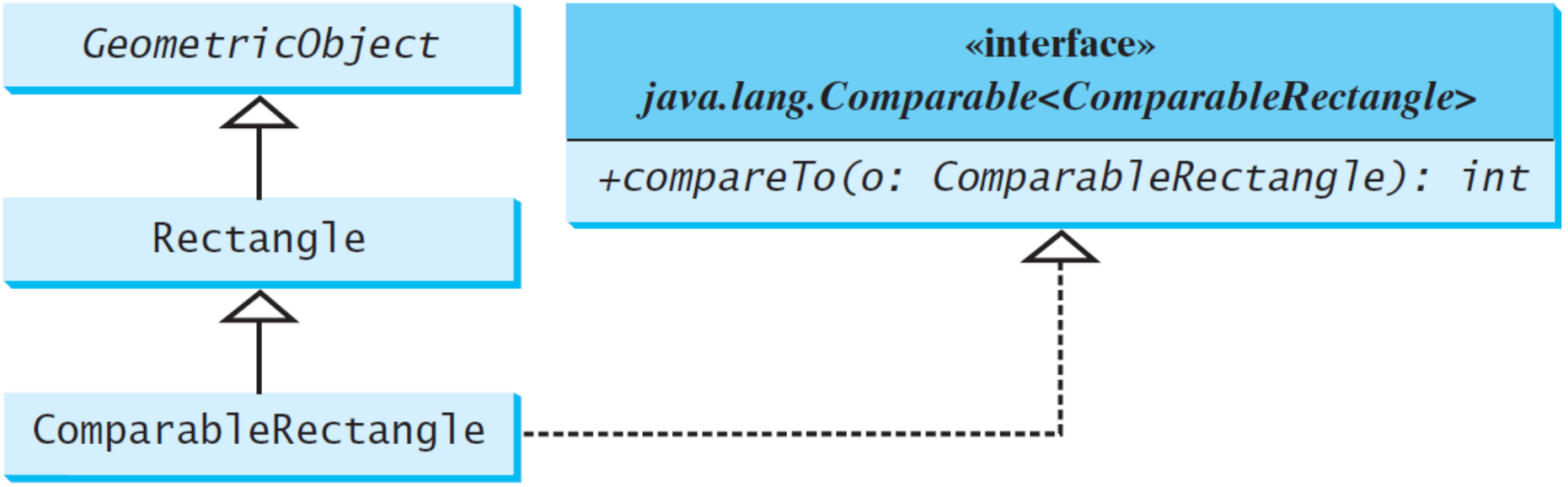
```
s instanceof String
s instanceof Object
s instanceof Comparable
```

```
d instanceof java.util.Date
d instanceof Object
d instanceof Comparable
```

Java.util.Arrays.sort (dizi) yöntemi, bir dizideki öğelerin Comparable <E> örnekleri olmasını gerektirir.

SortComparableObjects Run

Comparable Arayüzü implement için Sınıfların Tanımlanması



ComparableRectangle

SortRectangles

Run

`Cloneable` Arayüzleri

Marker Arayüzü: Boş bir arayüz.

Marker arayüzü sabitler veya yöntemler içermez. Bir sınıfın belirli istenen özelliklere sahip olduğunu belirtmek için kullanılır. Cloneable arayüzü uygulayan bir sınıf cloneable olarak işaretlenir ve nesneleri Object sınıfında tanımlanan clone () yöntemi kullanılarak klonlanabilir.

```
package java.lang;
public interface Cloneable {
}
```



Örnekle

Java kütüphanesinde birçok sınıf (ör. Date ve Calendar) Cloneable implemente eder. Böylece, bu sınıfların örnekleri de klonlanabilir. Örneğin, aşağıdaki kod

```
Calendar calendar = new GregorianCalendar(2003, 2, 1);
Calendar calendarCopy = (Calendar)calendar.clone();
System.out.println("calendar == calendarCopy is " +
  (calendar == calendarCopy));
System.out.println("calendar.equals(calendarCopy) is " +
  calendar.equals(calendarCopy));
```

displays

calendar == calendarCopy is false

calendar.equals(calendarCopy) is true



Cloneable Arayüzünün Implemente Edilmesi

Cloneable arayüzü uygulayan özel bir sınıf tanımlamak için, sınıfın Object sınıfındaki clone () yöntemini override edilmesi gerekir. Aşağıdaki kod, Cloneable ve Comparable implemente edilen House adlı bir sınıfı tanımlar.

House

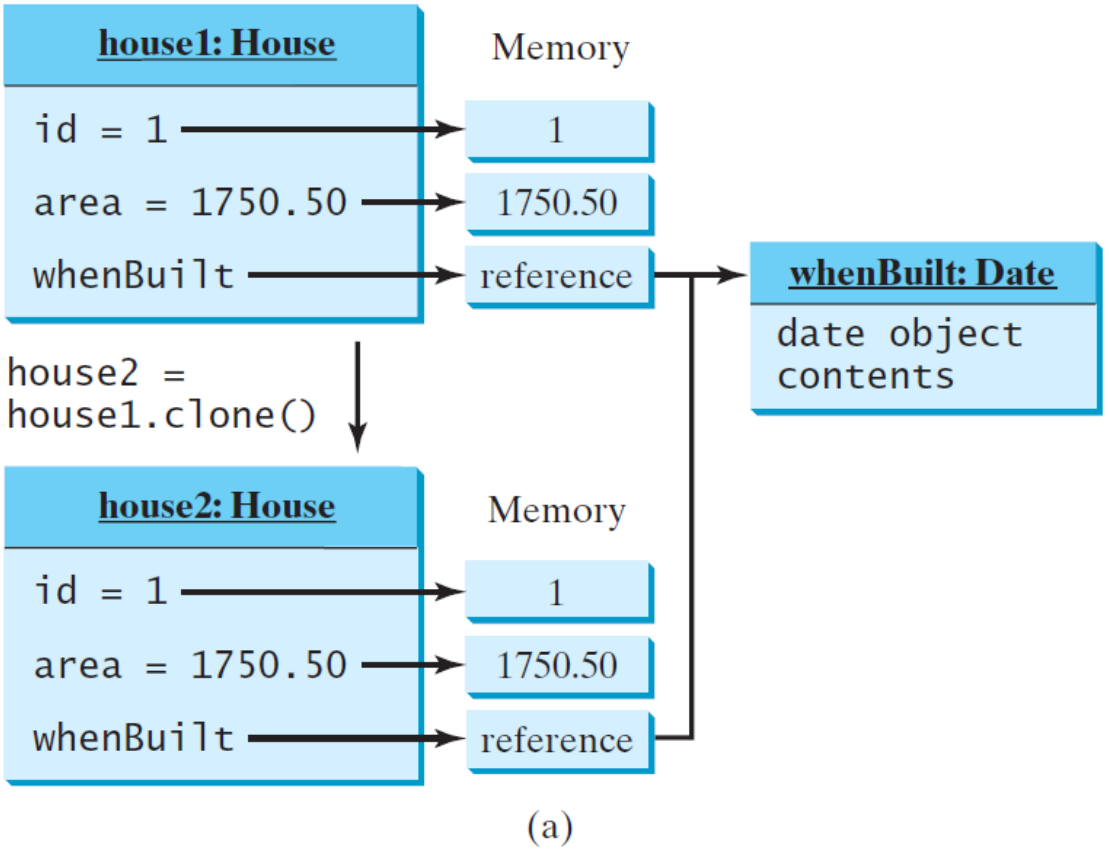


Shallow vs. Deep Kopya

House house1 = new House(1, 1750.50);

House house2 = (House)house1.clone();

Shallow Copy

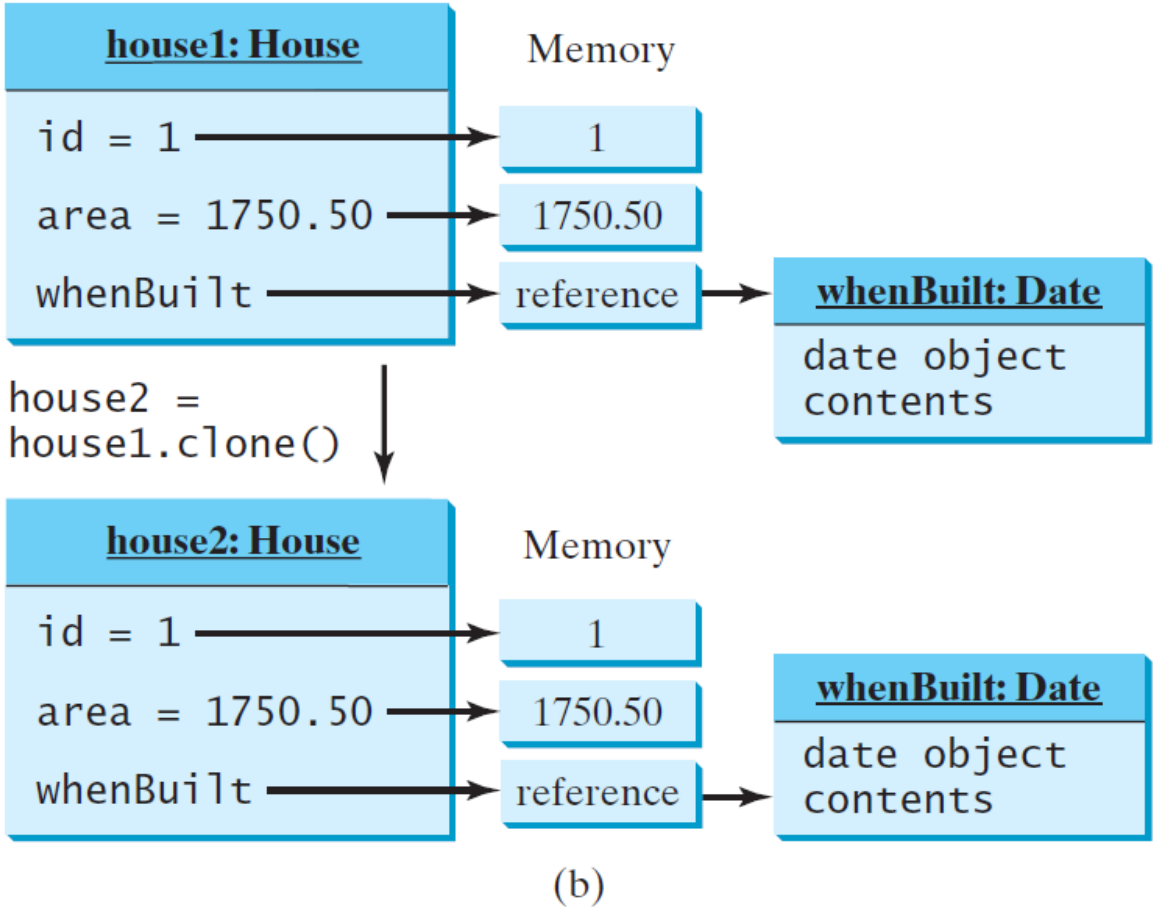


Shallow vs. Deep Copy

House house1 = new House(1, 1750.50);

House house2 = (House)house1.clone();

Deep Copy



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Arayüzler ve. Soyut Sınıflar

Bir arayüzde veriler sabit olmalıdır; soyut bir sınıf her türlü veriye sahip olabilir.

Bir arayüzdeki her yöntemin uygulama olmadan yalnızca imzası vardır; soyut bir sınıfın somut yöntemleri olabilir.

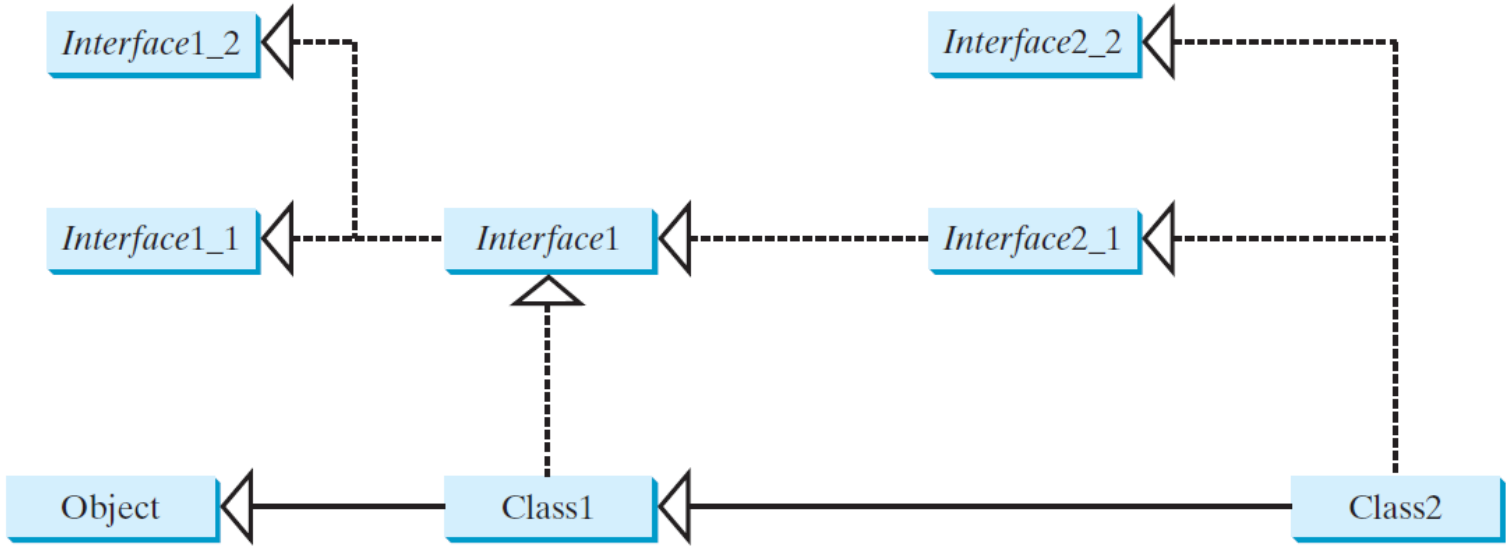
	Variables	*Constructors*	*Methods*
Abstract class	No restrictions.	Constructors are invoked by subclasses through constructor chaining. An abstract class cannot be instantiated using the new operator.	No restrictions.
Interface	All variables must be `public static final`.	No constructors. An interface cannot be instantiated using the new operator.	All methods must be public abstract instance methods



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Arayüzler ve. Soyut Sınıflar

Tüm sınıflar tek bir kök olan Object sınıfını paylaşır, ancak arayüzler için tek bir kök yoktur. Bir sınıf gibi, arayüz de bir tür tanımlar. Bir arayüz türünün değişkeni, arayüzü implemente eden sınıfın herhangi bir örneğine referans olabilir. Sınıf bir arayüzü extend ederse, bu arayüz bir üst sınıfla aynı rolü oynar. Bir arayüzü veri türü olarak kullanabilir ve arayüz türündeki bir değişken, alt sınıfa ya da tam tersine cast edilebilir.



Örneğin; c Sınıf2'nin bir örneğidir. c ayrıca Object, Class1, Interface1, Interface1_1, Interface1_2, Interface2_1 ve Interface2_2'nin bir örneğidir.

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Dikkat: arayüzlerin çakışması

Nadir durumlarda, bir sınıf çakışan bilgilerle iki arayüz uygulayabilir (örneğin, farklı değerlere sahip iki sabit veya aynı imzalı ancak farklı dönüş tipine sahip iki yöntem). Bu tür hatalar derleyici tarafından algılanacaktır.



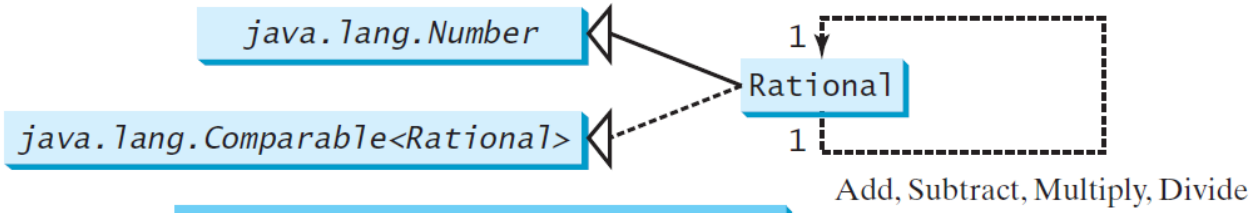
Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Bir arayüz mü/sınıf mı kullanılır?

Soyut sınıflar ve arayüzler ortak özellikleri modellemek için kullanılabilir. Bir arayüz veya sınıf mı kullanacağımıza nasıl karar verilir? Genel olarak, güçlü bir ebeveyn-çocuk ilişkisini açıkça tanımlayan bir ilişki sınıflar kullanılarak modellenmelidir. Örneğin, staff member bir personeldir.

Zayıf bir ilişki, aynı zamanda bir çeşit ilişkidir, bir nesnenin belirli bir özelliğe sahip olduğunu gösterir. Zayıf bir ilişki, arayüzler kullanılarak modellenebilir. Örneğin, tüm stringler karşılaştırılabilir olduğundan String sınıfı Comparable arayüzünü implemente eder. Birden fazla miras isteniyorsa, tek miras kısıtlamasını atlatmak için arayüzler kullanabilir. Birden fazla miras durumunda, birini bir süper sınıf, diğerini arayüz olarak tasarlamanız gerekir.

The `Rational` Sınıfı



Rational
-numerator: long -denominator: long
+Rational() +Rational(numerator: long, denominator: long) +getNumerator(): long +getDenominator(): long +add(secondRational: Rational): Rational +subtract(secondRational: Rational): Rational +multiply(secondRational: Rational): Rational +divide(secondRational: Rational): Rational +toString(): String -gcd(n: long, d: long): long

-numerator: long	The numerator of this rational number.
-denominator: long	The denominator of this rational number.
+Rational()	Creates a rational number with numerator 0 and denominator 1.
+Rational(numerator: long, denominator: long)	Creates a rational number with a specified numerator and denominator.
+getNumerator(): long	Returns the numerator of this rational number.
+getDenominator(): long	Returns the denominator of this rational number.
+add(secondRational: Rational): Rational	Returns the addition of this rational number with another.
+subtract(secondRational: Rational): Rational	Returns the subtraction of this rational number with another.
+multiply(secondRational: Rational): Rational	Returns the multiplication of this rational number with another.
+divide(secondRational: Rational): Rational	Returns the division of this rational number with another.
+toString(): String	Returns a string in the form "numerator/denominator." Returns the numerator if denominator is 1.
-gcd(n: long, d: long): long	Returns the greatest common divisor of n and d.

Rational TestRationalClass Run

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Sınıf Tasarımı

(Tutarlılık) Bir sınıf tek bir varlığı tanımlamalı ve tüm sınıf işlemleri tutarlı bir amacı desteklemek için mantıksal olarak birbirine uymalıdır. Örneğin, öğrenciler için bir sınıf kullanabilirsiniz, ancak öğrenciler ve personel aynı sınıfta birleştirilmemelidir, çünkü öğrenciler ve personel farklı varlıklara sahiptir.



Sınıf Tasarımı

(Sorumlulukları birbirinden ayırmak) Çok fazla sorumluluğu olan tek bir varlık, sorumlulukları ayırmak için birkaç sınıfa ayrılabilir. Örneğin String, StringBuilder ve StringBuffer sınıflarının tümü stringler ile ilgilenir, ancak farklı sorumlulukları vardır. String sınıfı değişmez stringler ile ilgilenir, StringBuilder sınıfı değiştirilebilir stringler oluşturmak içindir ve StringBuffer sınıfı, StringBuffer'ın stringleri güncellemek için senkronize yöntemler içermesi dışında StringBuilder'e benzer.

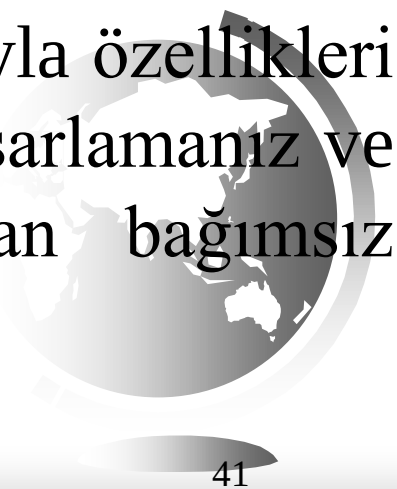


Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Sınıf Tasarımı

Sınıflar yeniden kullanım için tasarlanmıştır. Kullanıcılar sınıfları birçok farklı kombinasyon ve ortamda birleştirebilir. Bu nedenle,

- kullanıcının ne yapabileceği veya ne zaman yapabileceği konusunda herhangi bir kısıtlama getirmeyen bir sınıf tasarlanmalıdır
- kullanıcının herhangi bir değer kombinasyonuyla özellikleri istediği gibi ayarlayabilmesi için özellikleri tasarlamanız ve tasarım yöntemlerini gerçekleşme sırasından bağımsız olarak işlev görmesi gerekir.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Sınıf Tasarımı

Bir genel no-arg constructor oluşturulmalı ve mümkünse , equals yöntemi ve Object sınıfında tanımlanan toString yöntemi override edilmelidir.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Sınıf Tasarımı

- Standart Java programlama stilini ve adlandırma kurallarını uygulanmalıdır. Sınıflar, veri alanları ve yöntemler için tanımlayıcı adlar seçilmelidir.
- Veri bildirimini her zaman constructor önce ve constructor yöntemlerden önce yerleştirin. Programlama hatalarından kaçınmak için her zaman bir constructor sağlayın ve değişkenleri başlatın.



Görünürlük (Visibility) Değiştiricileri Kullanma

Her sınıf, biri sınıf kullanıcıları ve diğeri sınıfın genişleticileri için olmak üzere iki kontrat sunabilir.

Sınıf kullanıcılarına yönelikse alanları özel ve erişimci yöntemlerini herkese açık hale getirin.

Sınıf genişleticileri için tasarlanmışsa alanları veya yöntemi koruyun. Genişleticiler için sözleşme, kullanıcılar için sözleşmeyi kapsar. (Extend) Genişletilmiş sınıf, bir örnek yönteminin korumalıdan herkese görünürlüğünü artırabilir veya uygulamasını değiştirebilir, ancak uygulama asla bu sözleşmeyi ihlal edecek şekilde değiştirmemelidir.



Görünürlük (Visibility) Değiştiricileri Kullanma

Bir sınıf, verilerini istemciler tarafından doğrudan erişimden gizlemek için private keyword kullanmalıdır.

Kullanıcılara özel verilere erişim sağlamak için get ve set yöntemleri kullanılmalıdır. Ancak yalnızca kullanıcının görmesini veya değiştirmesini istediğiniz özel verilere erişilebilir.

Sınıf ayrıca istemci kullanımı için tasarlanmamış yöntemleri de gizlemelidir. Örneğin, Rational sınıfındaki gcd yöntemi özeldir, çünkü yalnızca sınıf içinde dahili kullanım içindir.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

static Modifier kullanma

Sınıfın tüm örnekleri tarafından paylaşılan bir özellik statik özellik olarak bildirilmelidir.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.